AI VIBE CODING COURSE · STUDENT PROJECT PACK

Tutoring Marketplace

A small, real-world web application that helps learners find tutors and request a tutoring session.

This file takes one business idea from prompt to build-ready requirements. Use it as a reference, then adapt the prompts for your own capstone.

PROJECT SIZE

Small MVP · 2 user roles · 5 core workflows

SUGGESTED BUILD

Responsive web app · relational database · email notifications

INCLUDED

Idea · 3 reusable prompts · PRD · functional requirements · technical requirements

OUT OF SCOPE

Payments · video calls · ratings · calendar sync · AI matching

1 · IDEA DESCRIPTION

A focused marketplace can validate demand without complex platform features

The product connects learners who need subject help with independent tutors. The first release proves only the essential loop: join, publish, discover, request, respond, and track.

Business problem

Learners often rely on scattered messages, community groups, and referrals to find suitable tutors. Tutors spend time repeating their subjects, rates, and availability to each lead.

Proposed solution

A lightweight marketplace where tutors publish searchable profiles and learners send a structured session request. Both sides can see the request status.

Primary users

User	Need	Core outcome
Learner	Find qualified help by subject, level, price, and format.	Send a clear request to a suitable tutor.
Tutor	Present an offer and manage incoming leads.	Accept or decline a request with one action.
Administrator	Keep the pilot usable and safe.	View users and deactivate a problematic profile.

Included in the MVP

- Email/password sign-up and sign-in
- Learner and tutor profiles
- Tutor directory with basic filters
- Session request form
- Accept, decline, cancel, complete
- In-app status view and basic email notices
- Minimal administrator view

Deferred

- Online payments and refunds
- Built-in chat or video
- Ratings and reviews
- Recurring bookings
- Calendar synchronization
- Identity or credential verification
- AI recommendations

Working assumptions. One tutor handles each request; prices are shown as tutor-entered hourly rates; payment happens outside the platform; all times are stored in Coordinated Universal Time (UTC) and displayed in the user’s timezone.

2 · PROMPT

Prompt for creating the product requirements document

Paste this into an AI assistant. Replace bracketed items if you want to change the market, geography, or technical boundary.

```
You are a senior product manager. Write a concise but build-ready Product Requirements Document
(PRD) for a small tutoring marketplace MVP.

BUSINESS CONTEXT
- Learners need a simple way to find tutors and request a session.
- Tutors need a simple profile and a place to accept or decline requests.
- The pilot is for a small community and must be realistic for a student team to build.

PRIMARY USERS
1. Learner
2. Tutor
3. Administrator with limited moderation access

REQUIRED MVP CAPABILITIES
- Email/password account creation and sign-in
- Learner profile and tutor profile
- Tutor directory searchable by keyword and filterable by subject, level, session format, and
maximum hourly rate
- Tutor profile page
- Session request with proposed date/time, duration, format, and note
- Tutor can accept or decline; learner can cancel; either side can mark an accepted session
complete
- Dashboard showing request status
- Basic email notifications for new request and status change
- Administrator can list users and deactivate/reactivate a profile
```

2 · PROMPT CONTINUED

Scope controls and output structure complete the PRD prompt

```
CONSTRAINTS
- Exclude payments, refunds, in-app chat, live video, ratings, recurring bookings, calendar
sync, credential verification, and AI matching.
- Payment and lesson delivery happen outside the application.
- Use one tutor per request.
- Store time in UTC and display it in the user's selected timezone.
- Keep the MVP suitable for a 2-4 person student team.

DOCUMENT STRUCTURE
1. Executive summary
2. Problem statement and opportunity
3. Goals and non-goals
4. Personas
5. End-to-end workflows
6. User stories grouped by role
7. Product requirements with IDs and priorities
8. Business rules and status lifecycle
9. Non-functional expectations stated in product language
10. Success metrics for a small pilot
11. Risks, assumptions, dependencies, and open questions
12. Two-phase roadmap: MVP and later enhancements

QUALITY RULES
- Make every requirement observable and testable.
- Label priorities Must, Should, or Could.
- Do not invent market statistics, legal claims, or integration commitments.
- Distinguish assumptions from confirmed decisions.
- Keep scope deliberately small and identify anything that could cause scope creep.
```

Student check. Before accepting the output, confirm that the document keeps the exclusions, names all three roles, defines booking statuses, and never quietly adds payments or chat.

3 · PROMPT

Prompt for writing functional requirements

Use this after the PRD is approved. Paste the PRD below the prompt so the requirements stay traceable to product decisions.

```
You are a business analyst. Convert the approved PRD below into functional requirements for a
small tutoring marketplace MVP.

OUTPUT FORMAT
A. Actors, permissions, and access boundaries
B. Functional requirements grouped by module:
   - Authentication
   - User profiles
   - Tutor discovery
   - Session requests
   - Dashboards and notifications
   - Administration
C. Business rules and validations
D. Booking status-transition table
E. Error and empty-state behavior
F. Acceptance criteria for each Must requirement
G. Traceability matrix from PRD requirement ID to functional requirement ID

REQUIREMENT STYLE
- Give every requirement a stable ID such as FR-AUTH-01.
- Write: “The system shall…” and describe one observable behavior.
- State actor, trigger, required data, validation, expected result, failure behavior, and
authorization where relevant.
- Use Given/When/Then acceptance criteria for Must requirements.
- Separate business rules from interface suggestions.
- Mark derived assumptions clearly; do not turn them into confirmed scope.
- Flag ambiguity as an open question instead of inventing an answer.

BOUNDARIES TO PRESERVE
- Two customer roles: learner and tutor; one limited administrator role.
- No payments, chat, video, ratings, recurring bookings, calendar sync, credential verification,
or AI matching.
- A user chooses one customer role at registration for the MVP.
- A request belongs to one learner and one tutor.
- Valid request statuses: Pending, Accepted, Declined, Cancelled, Completed.
- Tutors respond only to their own requests. Learners cancel only their own requests.
- Deactivated profiles cannot sign in or appear in search.

COMPLETENESS CHECK
At the end, list any PRD requirement that has no functional requirement and any functional
requirement that is not justified by the PRD.

APPROVED PRD
[PASTE THE APPROVED PRD HERE]
```

Strong output

Requirements are atomic, numbered, testable, authorized, and linked to a product need.

Weak output

Requirements mix several behaviors, prescribe screens without need, or add features that the PRD excluded.

4 · PROMPT

Prompt for writing technical requirements

Use the approved PRD and functional requirements as inputs. The prompt deliberately favors a simple architecture over a production-scale platform.

```
You are a pragmatic software architect. Write technical requirements for a small tutoring
marketplace MVP using the approved PRD and functional requirements below.

ARCHITECTURE BOUNDARY
- Recommend a modular monolith, not microservices.
- Use a responsive web client, one application API, one relational database, and one
transactional email provider.
- The solution must be deployable on a free or low-cost student-friendly platform.
- Keep vendor choices swappable. If you name a technology, explain the selection criterion
and provide one alternative.

REQUIRED SECTIONS
1. Architecture overview and component responsibilities
2. Technology decision table
3. Data model: entities, key fields, relationships, constraints, and indexes
4. API contract: method, path, actor, request, response, errors, and authorization
5. Authentication, authorization, input validation, privacy, and audit requirements
6. Timezone and date/time handling
7. Email notification design and failure handling
8. Performance, availability, accessibility, compatibility, observability, backup, and
recovery targets appropriate to a pilot
9. Environments, configuration, database migration, deployment, and rollback
10. Test strategy mapped to critical workflows
11. Technical risks, trade-offs, and open decisions
12. Implementation sequence based on dependencies

QUALITY RULES
- Give technical requirements stable IDs such as TR-SEC-01.
- Trace each major technical requirement to one or more functional requirement IDs.
- Define measurable targets without pretending they are legal or contractual service-level
agreements.
- Do not expose password hashes, internal errors, or personal data in logs.
- Enforce authorization on the server, not only in the interface.
- Use UTC for stored timestamps and an IANA timezone name for display preferences.
- Define idempotency or duplicate-request protection for create actions.
- Do not add excluded product features.
- End with a “minimum build” checklist that a student can implement in order.

APPROVED PRD
[PASTE PRD HERE]

APPROVED FUNCTIONAL REQUIREMENTS
[PASTE FUNCTIONAL REQUIREMENTS HERE]
```

> **Student check.** Reject an answer that introduces multiple services without a need, omits server-side authorization, or specifies a database without defining relationships and constraints.

5 · WORKED PRODUCT REQUIREMENTS DOCUMENT

The MVP proves the tutor-discovery-to-request loop

Executive summary

The Tutoring Marketplace is a responsive web application for a small community pilot. Learners browse tutor profiles and send structured session requests. Tutors publish their offer and respond to requests. A limited administrator can manage profile access. The MVP succeeds if real users can complete this loop without staff coordinating it manually.

Problem and opportunity

Informal tutor discovery often happens across unrelated posts and direct messages. Learners cannot compare offers consistently, while tutors repeatedly answer the same questions. A small directory plus booking-request workflow can create a shared, trackable process without taking on payment or lesson delivery.

Goals

- Make relevant tutors discoverable.
- Standardize the first session request.
- Give both parties one status record.
- Give tutors control over incoming requests.
- Give an administrator minimal access control.

Non-goals

- Collect or distribute money.
- Host messaging or lessons.
- Verify qualifications.
- Rank tutors by quality.
- Automate matching with AI.
- Replace a calendar system.

Personas

Persona	Context	Job to be done	Concern
Lena, learner	Needs math help for an upcoming course.	Compare relevant tutors and request a suitable time.	Whether the tutor covers her level, format, and budget.
Tariq, tutor	Teaches evenings online and weekends in person.	Show a clear offer and respond to qualified leads.	Incomplete requests and conflicting times.
Amina, administrator	Supports a small pilot community.	Find an account and deactivate misuse.	Acting without seeing more personal data than needed.

Success measures for the pilot

Measure	Definition	Pilot target
Profile completion	Activated tutors with all required searchable fields.	At least 80%
Request conversion	Learners who view a tutor and submit a request in the same visit.	Baseline and monitor; no invented benchmark
Response rate	Requests accepted or declined within 48 hours.	At least 70%
Workflow reliability	Valid requests that reach the intended tutor dashboard.	At least 99% during pilot tests

5 · PRODUCT REQUIREMENTS DOCUMENT

Five workflows define the complete first release

Create account › Complete profile › Find tutor › Request session › Resolve status

End-to-end workflows

1. **Join and profile:** a user chooses learner or tutor, verifies required fields, and completes a role-specific profile.
2. **Discover:** a learner searches active tutor profiles and applies subject, level, format, and maximum-rate filters.
3. **Request:** the learner proposes a future date/time, duration, format, and note to one tutor.
4. **Respond:** the tutor accepts or declines. The learner may cancel while the request is Pending or Accepted.
5. **Close:** either participant marks an Accepted request Completed after its scheduled start time.

User stories

ID	As a...	I want...	So that...
US-L01	Learner	to filter tutors	I can reduce an unfamiliar list to relevant choices.
US-L02	Learner	to review a full tutor profile	I can decide before sharing a request.
US-L03	Learner	to submit and cancel a request	I can manage my intent without staff help.
US-T01	Tutor	to publish my offer	learners understand my subjects, levels, format, and rate.
US-T02	Tutor	to accept or decline my requests	the learner receives a clear answer.
US-A01	Administrator	to deactivate a profile	an account can be removed from use and discovery.

Prioritized product requirements

ID	Priority	Requirement
PR-01	Must	The product supports secure account creation, sign-in, sign-out, and role-based access.
PR-02	Must	Tutors can create and update a searchable offer profile.
PR-03	Must	Learners can search and filter active tutor profiles.
PR-04	Must	Learners can create a valid session request for one tutor.
PR-05	Must	Authorized participants can perform permitted status changes.
PR-06	Must	Each role has a dashboard with relevant requests and status.
PR-07	Should	The product emails the affected party after a new request or status change.
PR-08	Must	An administrator can list, deactivate, and reactivate profiles.

5 · PRODUCT REQUIREMENTS DOCUMENT

Explicit rules keep booking status predictable

Status lifecycle

Current	Allowed next state	Actor	Condition
Pending	Accepted	Assigned tutor	Request is active and scheduled in the future.
Pending	Declined	Assigned tutor	No further reason is required in MVP.
Pending	Cancelled	Requesting learner	Request belongs to learner.
Accepted	Cancelled	Requesting learner	Cancellation remains possible in MVP.
Accepted	Completed	Learner or tutor	Scheduled start time has passed.
Declined, Cancelled, Completed	None	None	Terminal states are immutable.

Business rules

- **BR-01:** Each account has one customer role in MVP.
- **BR-02:** Only active tutor profiles appear in discovery.
- **BR-03:** A session request references one learner and one tutor.
- **BR-04:** Proposed time must be in the future.
- **BR-05:** Duration is 30, 45, 60, 90, or 120 minutes.
- **BR-06:** Format must be online or in person.
- **BR-07:** Server-side authorization controls every transition.
- **BR-08:** Deactivation blocks sign-in and discovery.

5 · PRODUCT REQUIREMENTS DOCUMENT

Quality boundaries keep the pilot honest

Product-level quality expectations

- Core pages work on current mobile and desktop browsers and remain keyboard usable.
- A typical directory search should feel responsive under pilot load.
- Users receive clear validation messages without exposing system internals.
- Personal data is collected only for the stated workflow; public profiles exclude email addresses.
- Failure to send email does not reverse a successful booking status change.

Roadmap

MVP	Later, only after validation
Identity, role profiles, directory, filters, requests, status dashboard, basic email, admin deactivation.	Password reset, availability windows, in-app messaging, calendar sync, payments, reviews, verified credentials, AI matching.

Risks, dependencies, and open questions

Type	Item	Response
Risk	Stale tutor availability creates poor requests.	Show a free-text availability summary; add structured slots later.
Risk	Email delivery is delayed.	Keep dashboard as the source of truth.
Dependency	Transactional email provider.	Abstract sending behind one application service.
Open	Should tutors show city for in-person lessons?	Decide before finalizing profile privacy fields.

6 · WORKED FUNCTIONAL REQUIREMENTS

Access follows role, ownership, and account status

Actors and permissions

Action	Learner	Tutor	Administrator
Manage own profile	Yes	Yes	No
Browse active tutors	Yes	Optional read-only	Yes
Create request	Yes	No	No
Accept or decline request	No	Own assigned requests	No
Cancel request	Own requests	No	No
Deactivate profile	No	No	Yes

Authentication

ID	The system shall...	Validation / failure behavior
FR-AUTH-01	create an account from email, password, display name, role, and timezone.	Email is unique after normalization; role is Learner or Tutor; invalid fields return field-level errors.
FR-AUTH-02	authenticate an active account and begin a session.	Invalid credentials return a generic error; deactivated accounts are denied.
FR-AUTH-03	end the current authenticated session.	Session token or cookie becomes unusable.
FR-AUTH-04	authorize each protected action by role and ownership.	Unauthenticated requests return 401; authenticated but forbidden requests return 403.

Profiles

ID	The system shall...	Required behavior
FR-PRO-01	let a learner update display name, timezone, learning level, and optional goals.	Only the owner may read or update non-public learner details.
FR-PRO-02	let a tutor update bio, subjects, levels, formats, hourly rate, currency, and availability summary.	At least one subject, level, and format are required before publication.
FR-PRO-03	publish an active, complete tutor profile in discovery.	Public response excludes email and internal identifiers not needed by the client.

Discovery

ID	The system shall...	Required behavior
FR-SRCH-01	list active, published tutors with pagination.	Default sort is newest profile; empty result explains how to clear filters.
FR-SRCH-02	filter by subject, level, format, and maximum hourly rate.	Filters combine with AND; multiple subject values combine with OR.
FR-SRCH-03	search display name, bio, and subject labels by keyword.	Blank keyword behaves as no keyword filter.

6 · FUNCTIONAL REQUIREMENTS

Session requests preserve ownership and valid transitions

Session request management

ID	The system shall...	Validation / authorization
FR-REQ-01	let a learner create a Pending request for one active tutor.	Requires future start, allowed duration, supported format, and note of 20-1,000 characters.
FR-REQ-02	prevent accidental duplicate creation from a repeated submission.	Repeated idempotency key returns the original result without a second request.
FR-REQ-03	let the assigned tutor accept or decline a Pending request.	Any stale or invalid transition returns conflict and leaves status unchanged.
FR-REQ-04	let the requesting learner cancel a Pending or Accepted request.	Other users are forbidden; terminal statuses remain unchanged.
FR-REQ-05	let either participant complete an Accepted request after its start time.	Completion before start returns a validation error.
FR-REQ-06	record status, actor, and timestamp for each successful transition.	History is append-only through ordinary application actions.

Dashboards and notifications

ID	The system shall...	Required behavior
FR-DASH-01	show a learner only requests they created.	List includes tutor, proposed time in viewer timezone, format, and status.
FR-DASH-02	show a tutor only requests assigned to them.	Pending requests appear first, followed by newest update time.
FR-NOT-01	queue an email after request creation or status change.	Recipient is the affected party; email failure is logged and does not roll back the change.

Administration

ID	The system shall...	Required behavior
FR-ADM-01	let an administrator list and search accounts by display name or email.	Results show role and active status; access is admin-only.
FR-ADM-02	let an administrator deactivate or reactivate an account with a reason.	Action records admin, account, reason, previous state, new state, and time.

Error and empty-state behavior

- Validation failures identify the affected field and keep non-sensitive user input available for correction.
- Not-found responses do not reveal whether an inaccessible private record exists.
- Conflict responses explain that the record changed and invite a refresh.
- Empty tutor search and empty dashboard states explain the next available action.
- Unexpected errors show a safe message and a support reference, never a stack trace.

6 · FUNCTIONAL REQUIREMENTS

Acceptance criteria make every Must requirement testable

PRD / functional IDs	Acceptance criterion
PR-01 FR-AUTH-01–04	**Given** a visitor supplies a unique valid email, password, display name, role, and timezone, **when** registration succeeds, **then** an active account is created and protected actions enforce its selected role.
PR-02 FR-PRO-02–03	**Given** an authenticated tutor completes all required offer fields, **when** the profile is published, **then** it appears in discovery without email or private account fields.
PR-03 FR-SRCH-01–03	**Given** published active tutors exist, **when** a learner applies subject, level, format, and maximum-rate filters, **then** every result satisfies all selected filter groups.
PR-04 FR-REQ-01–02	**Given** a learner submits a valid future request to an active tutor, **when** the create action succeeds, **then** one Pending request is visible to both parties and a repeated identical submission does not create another.
PR-05 FR-REQ-03–06	**Given** a Pending request, **when** its assigned tutor accepts it, **then** status becomes Accepted, the transition is recorded, and an unrelated tutor receives a forbidden response for the same action.
PR-06 FR-DASH-01–02	**Given** a user has multiple requests, **when** they open the dashboard, **then** only requests they own or are assigned to appear, with time shown in their selected timezone.
PR-08 FR-ADM-01–02	**Given** an administrator deactivates a tutor with a reason, **when** the action succeeds, **then** the tutor cannot sign in, disappears from discovery, and the administrative action is recorded.

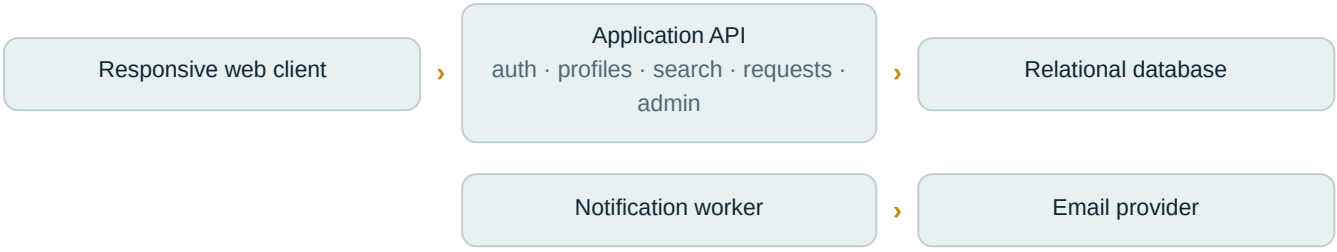
Traceability check

Product requirement	Functional coverage	Status
PR-01 Identity and access	FR-AUTH-01–04	Covered
PR-02 Tutor offer	FR-PRO-02–03	Covered
PR-03 Discovery	FR-SRCH-01–03	Covered
PR-04 Request creation	FR-REQ-01–02	Covered
PR-05 Status actions	FR-REQ-03–06	Covered
PR-06 Dashboards	FR-DASH-01–02	Covered
PR-07 Email	FR-NOT-01	Covered
PR-08 Administration	FR-ADM-01–02	Covered

Completeness result. No approved product requirement lacks functional coverage. No worked functional requirement introduces an unapproved feature.

7 · WORKED TECHNICAL REQUIREMENTS

A modular monolith is enough for the pilot



Component responsibilities

Component	Responsibility	Boundary
Web client	Forms, discovery, dashboards, accessible status feedback.	Never decides authorization or final validity.
Application API	Authentication, policy enforcement, business rules, persistence, email job creation.	One deployable application with internal modules.
Relational database	Accounts, profiles, catalog labels, requests, history, admin audit, email jobs.	Constraints protect identity, ownership, and valid references.
Notification worker	Claims queued email jobs, sends, retries transient failures, records outcome.	No booking change depends on delivery success.

Technology decision table

Layer	Recommended criterion	Example	Alternative
Client + API	One language, server rendering or API routes, strong form support.	Next.js with TypeScript	React + Express
Database	Relational constraints, transactions, portable SQL.	PostgreSQL	SQLite for local demo only
Data access	Typed schema and repeatable migrations.	Prisma ORM	Drizzle ORM
Authentication	Proven session handling; secure cookie support.	Framework auth library	Managed authentication service
Email	Transactional API, sandbox mode, delivery logs.	Provider adapter	SMTP adapter
Hosting	Managed HTTPS, environment variables, logs, rollback.	Student-friendly platform	Small container host

Key technical requirements

- **TR-ARC-01** The application shall deploy as one web service plus one worker process, sharing one relational database.
- **TR-ARC-02** Internal modules shall separate authentication, profiles, discovery, requests, notifications, and administration.
- **TR-DATA-01** Database migrations shall be versioned, reviewed, and applied before compatible application code.
- **TR-TIME-01** Stored instants shall use UTC; user preference shall use an IANA timezone name such as `America/Toronto`.

7 · TECHNICAL REQUIREMENTS

The data model makes role and booking rules explicit

Entity	Key fields	Constraints and indexes
User	id, email_normalized, password_hash, display_name, role, timezone, is_active, created_at	Unique email; role enum Learner/Tutor/Admin; index active status and role.
LearnerProfile	user_id, learning_level, goals	One-to-one with a Learner user; cascade restricted while requests exist.
TutorProfile	user_id, bio, hourly_rate_minor, currency, availability_summary, is_published	One-to-one with Tutor; nonnegative rate; index publication and rate.
Subject	id, name, slug	Unique name and slug; seeded catalog.
TutorSubject	tutor_id, subject_id	Composite primary key; indexes both directions.
TutorLevel	tutor_id, level	Composite unique pair; controlled level values.
TutorFormat	tutor_id, format	Composite unique pair; Online or InPerson.
SessionRequest	id, learner_id, tutor_id, starts_at_utc, duration_minutes, format, note, status, created_at, updated_at	Foreign keys to valid users; duration check; indexes tutor/status and learner/updated_at.
RequestHistory	id, request_id, from_status, to_status, actor_user_id, occurred_at	Append-only application path; index request/time.
IdempotencyKey	user_id, key, request_hash, response_ref, expires_at	Unique user/key; reject same key with different payload.
EmailJob	id, template, recipient_user_id, payload_json, state, attempts, next_attempt_at	Index state/next attempt; payload excludes secrets.
AdminAudit	id, admin_id, target_user_id, action, reason, before_json, after_json, occurred_at	Append-only application path; index target/time.

Relationship summary

- A User has zero or one role-specific profile. The application validates that profile type matches the account role.
- A TutorProfile has many Subjects, Levels, and Formats through constrained join tables.
- A SessionRequest belongs to one learner User and one tutor User and has many RequestHistory records.
- User deletion is not part of MVP. Deactivation preserves request and audit history.

Data rules

- **TR-DATA-02** Money shall be stored as integer minor units plus ISO currency code; it is display-only in this MVP.
- **TR-DATA-03** Status changes and their history entry shall commit in one database transaction.
- **TR-DATA-04** Public tutor queries shall select an explicit allowlist of fields rather than serialize account records.

7 · TECHNICAL REQUIREMENTS

The API exposes only the workflows the MVP needs

Method and path	Actor	Purpose	Key results
`POST /api/auth/register`	Public	Create customer account.	201; 422 invalid; 409 email exists.
`POST /api/auth/session`	Public	Sign in active user.	200 + secure session; 401 generic denial.
`GET /api/tutors`	Authenticated	Paginated search and filters.	200 + public profile summaries.
`GET /api/tutors/:id`	Authenticated	Read public tutor profile.	200; 404 absent or unavailable.
`PUT /api/tutor-profile`	Tutor	Create or update own offer.	200; 422 invalid; 403 wrong role.
`POST /api/requests`	Learner	Create Pending request.	201; 422 invalid; 409 duplicate/conflict.
`GET /api/requests`	Learner/Tutor	List requests visible to caller.	200 + role-filtered, paginated list.
`POST /api/requests/:id/transition`	Participant	Apply allowed target status.	200; 403 forbidden; 409 stale transition.
`GET /api/admin/users`	Admin	Search account summaries.	200; 403 non-admin.
`POST /api/admin/users/:id/status`	Admin	Activate or deactivate with reason.	200 + audit record; 422 invalid.

Representative create-request contract

REQUEST

`Headers: Idempotency-Key`

```
{ tutorId, startsAt, durationMinutes,
format, note }
```

The server derives learnerId from the authenticated session.

RESPONSE

```
{ id, tutor, startsAt, durationMinutes,
format, status: "Pending", createdAt }
```

The response contains only fields the caller may view.

Interface requirements

- **TR-API-01** JSON endpoints shall use consistent error objects: code, user-safe message, fieldErrors when applicable, and request reference.
- **TR-API-02** List endpoints shall cap page size and return pagination metadata.
- **TR-API-03** Mutation endpoints shall validate content type, body shape, field length, enum values, and authorization before changing data.
- **TR-API-04** Request creation shall accept an idempotency key and replay the first successful response for the same user and payload.
- **TR-API-05** Status transitions shall use current status in the update condition to prevent concurrent double transitions.

7 · TECHNICAL REQUIREMENTS

Security and operations stay modest but concrete

Security, privacy, and audit

ID	Requirement	Trace
TR-SEC-01	Passwords shall use a maintained adaptive password-hashing library; raw passwords shall never be logged or stored.	FR-AUTH-01–02
TR-SEC-02	Browser sessions shall use Secure, HttpOnly, SameSite cookies and rotate after sign-in.	FR-AUTH-02–03
TR-SEC-03	Every protected endpoint shall enforce role, ownership, and active-account policy on the server.	FR-AUTH-04
TR-SEC-04	Sign-in and write endpoints shall be rate-limited; all input shall be validated before use.	FR-AUTH-02; FR-REQ-01
TR-SEC-05	Logs shall exclude password hashes, session values, full notes, and unnecessary email addresses.	All
TR-AUD-01	Administrative status changes and booking transitions shall produce immutable application audit records.	FR-REQ-06; FR-ADM-02

Pilot quality targets

Area	Target for pilot validation
Performance	95th-percentile API response under 800 ms for ordinary reads and writes, excluding email delivery, at 25 concurrent users in a controlled test.
Availability	Best-effort managed hosting; publish planned maintenance. No contractual service-level agreement.
Accessibility	Keyboard-operable core flows, visible focus, labeled inputs, semantic headings, and WCAG 2.1 AA contrast checks.
Compatibility	Latest two major versions of Chrome, Edge, Firefox, and Safari; responsive at 360 px and wider.
Observability	Structured request logs, error tracking, email job states, health endpoint, and correlation/request reference.
Backup	Daily managed database backup retained for seven days; document and rehearse restore before pilot launch.
Recovery	Pilot objective: restore service and latest daily backup within four hours after a confirmed data-loss event.

Deployment and change control

- **TR-OPS-01** Development, test, and production shall use separate databases and environment-specific secrets.
- **TR-OPS-02** Secrets shall come from the hosting platform’s secret store, never source control or client bundles.
- **TR-OPS-03** Continuous integration shall run formatting, type checks, unit tests, and integration tests before deployment.
- **TR-OPS-04** Deployment shall apply backward-compatible migrations, verify health, then release traffic. Rollback shall restore the previous application version; destructive migrations require a forward-fix plan.

8 · BUILD SEQUENCE AND READINESS

Build in dependency order to avoid rework

1. ## Foundation

   Initialize repository, environments, database connection, migration process, shared validation, error format, and health endpoint.

2. ## Identity and authorization

   Implement account registration, sessions, role checks, active-account checks, and test fixtures for all three roles.

3. ## Profiles and catalogs

   Create learner and tutor profile models, seed subjects, validate publishable tutor profiles, and expose public field allowlists.

4. ## Discovery

   Build tutor directory queries, filters, pagination, tutor detail, empty states, and responsive result views.

5. ## Session requests

   Add create-request idempotency, status transitions, transactional history, role dashboards, and concurrency tests.

6. ## Notifications and administration

   Add queued email delivery, retry handling, user search, deactivation, reactivation, and audit records.

7. ## Hardening and pilot

   Run accessibility, security, browser, performance, backup-restore, and end-to-end acceptance checks; seed only clearly marked demo records.

8 · BUILD SEQUENCE AND READINESS

A narrow test plan defines “done”

Minimum test set

AUTOMATED

- Unit: validators and transition policy
- Integration: authorization and database constraints
- API: idempotency and conflict behavior
- End-to-end: learner request and tutor response

MANUAL

- Keyboard-only core workflow
- Mobile and desktop layouts
- Email failure without data rollback
- Backup restore rehearsal

Student readiness checklist

- ☐ I can explain the problem and the two-sided value.
- ☐ I can name what is deliberately excluded.
- ☐ I can trace each Must product requirement to functional behavior.
- ☐ I can explain the booking state transitions and who may perform them.
- ☐ I can sketch the four-component architecture and core data relationships.
- ☐ I can implement the seven build stages in order and demonstrate the acceptance flow.

> **Definition of done.** A learner can find a published tutor, submit one valid request, and see its status; the assigned tutor can accept or decline it; unauthorized users cannot act on it; the state change survives refresh and remains correct if email fails.